

Motivation

SVMs answer the question: Among all the hyperplanes that correctly separate two classes of data, which one is the best?

Hyperplanes

Recall from logistic regression that a linear classifier makes decisions based on the sign of $w^T x + b$. Points where $w^T x + b > 0$ are classified as positive, and negative when < 0 . Thus, the set of points where $w^T x + b = 0$ define a decision boundary. This hyperplane is fully specified by its normal vector w and its offset from the origin b .

The signed distance vector from a point x to the hyperplane is:

$$\rightarrow D = \frac{w^T x + b}{\|w\|}$$

Where the sign tells you which side of the hyperplane the point is on, and the magnitude tells you how far away it is.

Before we continue, note that if (w, b) defines a hyperplane, so do $(2w, 2b)$, $(3w, 3b)$ and all real

non-zero scalar multiples. This means we can freely rescale w and b without changing the decision boundary.

The Maximum Margin Classifier

Our goal is to find the hyperplane that sits as far as possible from all training points. A hyperplane that passes very close to some training points is risky, a small perturbation in the data, or a new point near the boundary could cause misclassification.

The margin of a classifier is the distance from the decision boundary to the nearest training points on either side, but its sign tells us if the point was classified correctly.

$$\gamma^{(i)} = \frac{y^{(i)} (w^T x^{(i)} + b)}{\|w\|}$$

Where $y^{(i)} \in \{-1, 1\}$ is the class label for point i . Since the sign of the distance tells us which side of the boundary the point is on, is $y^{(i)}$ and the distance have the same sign, the point was classified correctly.

Our goal is to maximise the margin of the closest point to the hyperplane:

$$\text{maximise } \gamma = \min_i \left(\frac{y^{(i)} (w^T x^{(i)} + b)}{\|w\|} \right)$$

Because of that min, trying to differentiate this would be very awkward, so we use a trick. Since we can scale w and b however we want, we choose it so that, for the closest point to the hyperplane:

$$y^{(i)} (w^T x^{(i)} + b) = 1$$

Thus, under this scaling, the margin of the closest point becomes

$$\gamma = \frac{1}{\|w\|}$$

and so minimising this is the same as minimising $\|w\|$.

Now $\|w\| = \sqrt{w^T w}$, which is not differentiable at 0. Therefore, it has gradient

$$\frac{\partial}{\partial w} \|w\| = \frac{w}{\|w\|}$$

which is messy to work with

We use another trick. Since $\|w\| \geq 0$, minimising $\|w\|$ is the same as minimising $\|w\|^2$, which differentiates much more clearly. We also add a factor of $1/2$ in front, just so the constants cancel.

$$\frac{\partial}{\partial w} \frac{1}{2} \|w\|^2 = w$$

Nice and clean. So the full optimisation problem is:

minimise $\frac{1}{2} \|w\|^2$ subject to

$$y^{(i)} (w^T x^{(i)} + b) \geq 1 \text{ for all } i$$

Recall that we normalised it so that $y^{(i)} (w^T x^{(i)} + b) = 1$ for the closest point.

Thus, that ≥ 1 constraint says that all the other points must be further away, or the same distance at best. Since this is positive, it also enforces all points being classified correctly.

This is a convex, constrained optimisation problem, a well studied class of problem.

The Margin Boundaries and Support Vectors

Under the canonical normalisation, there are two margin boundaries:

$$\begin{aligned} w^T x + b &= +1 & (\text{positive margin boundary}) \\ w^T x + b &= -1 & (\text{negative margin boundary}) \end{aligned}$$

The distance between them is $2 \times \frac{1}{\|w\|} = \frac{2}{\|w\|}$. The decision boundary $w^T x + b = 0$ lies exactly in the middle.

The training points that lie exactly on the margin boundaries satisfying $y^{(i)} (w^T x^{(i)} + b) = 1$ are called the support vectors. Notably, these are the only points that matter for defining the hyperplane. All other training

points could be removed without changing the solution. Thus, the SVM solution depends only on a small subset of the training data.

Lagrangian Duality

Recall how to solve convex optimisation problems via duality.

We construct our Lagrangian:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i [y^{(i)} (w^T x^{(i)} + b) - 1]$$

Where we have on $\alpha_i \geq 0$ for each constraint. Thus, if a constraint is violated, the term inside the sum is negative, then becomes positive by the $-$ out in front. This drives $\mathcal{L}$ upward, punishing infeasible solutions. To solve the dual problem, recall that maximising the unconstrained minimum as a function of α gives the primal optimum for convex problems.

1. Minimise $\mathcal{L}(w, b, \alpha)$ over w and b

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_i \frac{\partial}{\partial w} (\alpha_i (y^{(i)} (w^T x^{(i)} + b) - 1))$$

$$= w - \sum_i \alpha_i y^{(i)} x^{(i)}$$

$$\text{Set } \frac{\partial \mathcal{L}}{\partial w} = 0$$

$$\Rightarrow w = \sum_i \alpha_i y^{(i)} x^{(i)}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_i \alpha_i y^{(i)}$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y^{(i)} = 0$$

2. Sub back into $\mathcal{L}$:

$$\mathcal{L}(\alpha) = \frac{1}{2} \|\omega\|^2 - \sum_i \alpha_i y^{(i)} (\omega^T x^{(i)}) + \sum_i \alpha_i$$

Since $\|\omega\|^2 = \omega^T \omega$

$$\|\omega\|^2 = \left(\sum_i \alpha_i y^{(i)} x^{(i)} \right)^T \left(\sum_j \alpha_j y^{(j)} x^{(j)} \right)$$

Note we use two different indices for the copies of the sum, to keep them separate. If we just used i , we would only be along the diagonal.

$$= \sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

The scalars $\alpha_i, \alpha_j, y^{(i)}, y^{(j)}$ all factor out.

So the first term $\frac{1}{2} \|\omega\|^2$ is:

$$(1/2) \sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

Now for the $\omega^T x$:

$$\omega^T x^{(i)} = \left(\sum_j \alpha_j y^{(j)} x^{(j)} \right)^T x^{(i)} = \sum_j \alpha_j y^{(j)} x^{(j)T} x^{(i)}$$

So the middle term $-\sum_i \alpha_i y^{(i)} (\omega^T x^{(i)})$ becomes:

$$\sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

(Note: $x^{(i)T} x^{(i)} = x^{(i)T} x^{(i)}$ by symmetry)

All together:

$$\mathcal{L} =$$

$$(1/2) \sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

$$- \sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)} + \sum_i \alpha_i$$

The first two terms have the same double sum:

$$\frac{1}{2} S - S = -\frac{1}{2} S$$

So:

$$\mathcal{L}(\alpha) = \sum_i \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

3. Now maximise $\mathcal{L}(\alpha)$ w.r.t α
Formally:

$$\text{Maximise } \mathcal{L}(\alpha) = \dots \text{ subject to } \alpha_i \geq 0 \text{ and } \sum_i \alpha_i y^{(i)} = 0$$

This is another convex optimisation problem.

However, this has no closed solution, so we let a computer handle it for us. At the optimum, **most** $\alpha_i = 0$, these correspond to non-support vector points that lie outside or on the margin. Only the support vectors have $\alpha_i > 0$.

So, the computer has given us the optimal values of α_i . We now plug these back into our expression:

$$\omega = \sum_i \alpha_i y^{(i)} x^{(i)}$$

to get the optimal weights. There are some additional conditions to solving a constrained optimisation problem, called the **KKT Conditions**. You don't need to know much about these, except that they include a condition called **complementary slackness**, which says:

$$\alpha_i [y^{(i)} (\omega^T x^{(i)} + b) - 1] = 0$$

Intuitively, this should make sense.

Case 1: $\alpha_i = 0$

The multiplier is zero, corresponding to a non-support vector. This is most of the training points.

Case 2:

$y^{(i)} (\omega^T x^{(i)} + b) = 1$. The point lies on the margin boundary. These are the support vectors.

Why is this useful? Recall that we already knew that the support vectors satisfied $y^{(i)} (\omega^T x^{(i)} + b) = 1$, by the canonical normalisation.

However, complementary slackness gives us a more powerful picture.

If $\alpha_i > 0$, the point must be on the margin boundary. $\alpha_i = 0$ otherwise.

This tells us which α_i can be 0 and which necessarily cannot. This tells which training points correspond to support vectors. Once we know this, we can easily calculate b . Support vectors satisfy:

$$y^{(sv)} (\omega^T x^{(sv)} + b) = 1$$

$$\omega^T x^{(sv)} + b = 1/y^{(sv)}$$

Since $y^{(sv)} \in \{-1, +1\}$, $1/y^{(sv)} = y^{(sv)}$

$$b = y^{(sv)} - \omega^T x^{(sv)}$$

Any support vector gives the same b in theory. In practice, numerical errors mean we get slightly different values for each support vector. We just average:

$$b = \frac{1}{N_{sv}} \sum_{N_{sv}} y^{(sv)} - \omega^T x^{(sv)}$$

Complementary slackness also shows why SVM is sparse. Because most α_i are 0, the entire decision boundary is generated by a small subset of training data. This is highly practical at test time, since you only need to consider the support vectors for prediction, rather than a huge dataset.

Making Predictions

Given a point x , the prediction is

$$y = \text{Sign}(\omega^T x + b)$$

$$= \text{Sign} \left(\sum_i \alpha_i \cdot y^{(i)} (x^{(i)T} x) + b \right)$$

Soft-Margin SVM

Hard-Margin SVM requires the data to be linearly separable. If any point lies on the wrong side of the margin, the problem is infeasible, and no solution exists. Real data is rarely perfectly separable due to noise, outliers or genuine class overlap. We need a way to allow some violations.

Slack Variables

The soft-margin SVM introduces slack variables $\xi_i \geq 0$, one per training point, that measure how much each point violates the margin constraint:

$$y^{(i)} (\omega^T x^{(i)} + b) \geq 1 - \xi_i \quad \forall i, \xi_i \geq 0$$

Recall that, under the canonical scaling, a point on the margin satisfies

$$y^{(i)} (\omega^T x^{(i)} + b) = 1$$

≥ 1 means on the margin or further away. Thus when:

- $\xi_i = 0$ the point is correctly classified and outside/on the margin
- $0 < \xi_i \leq 1$ point is correctly classified but inside the margin
- $\xi_i > 1$ the point is misclassified

The soft margin optimisation problem penalises slack:

$$\min_{\omega, b, \xi} \frac{1}{2} \|\omega\|^2 + C \sum_i \xi_i$$

Subject to $y^{(i)} (\omega^T x^{(i)} + b) \geq 1 - \xi_i, \xi_i \geq 0$

So we see that large slack is penalised. The hyperparameter $C > 0$ controls the trade-off between maximising the margin (small $\|\omega\|^2$) and minimising constraint violations

(small ξ_i). Large C penalises violations heavily, the model tries hard to classify everything correctly, potentially with a smaller margin. Small C tolerates more violations in exchange for a larger margin.

Hinge Loss

Recall the soft margin optimisation:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

Subject to $y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \xi_i \geq 0$

We want to eliminate ξ_i and write this as an unconstrained problem. For fixed w and b , we want to minimise the objective over ξ_i . Since it only appears in the penalty term, we want each ξ_i as small as possible. However, ξ_i is constrained by two things:

- $\xi_i \geq 0$
- $\xi_i \geq 1 - y^{(i)}(w^T x^{(i)} + b)$

Where the second one comes from rearranging the margin constraint. So the smallest ξ_i can be is the larger $\Rightarrow$ the two bounds:

$$\xi_i = \max(0, 1 - y^{(i)}(w^T x^{(i)} + b))$$

Let's call the margin of point i m_i :

$$m_i = y^{(i)}(w^T x^{(i)} + b)$$

Then $\xi_i = \max(0, 1 - m_i)$

Three cases:

- $m_i \geq 1$, point is outside or on the margin boundary. $1 - m_i \leq 0$, so $\xi_i = 0$. No penalty
- $0 < m_i < 1$, point is correctly classified but inside the margin $1 - m_i > 0$, so $\xi_i = 1 - m_i$ Small penalty.

- $m_i < 0$, point is misclassified. $1 - m_i > 1$ so $\xi_i = 1 - m_i > 1$ large penalty.

Plugging this back in, we have:

$$\min \left(\frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y^{(i)}(w^T x^{(i)} + b)) \right)$$

We call this the Hinge Loss.

The Kernel Trick

A linear SVM finds a hyperplane in the original input space. However, this only works when classes are linearly separable, there exists a flat boundary that clearly divides them. But many real problems are not linearly separable, such as two concentric rings of points.

How do we still find a boundary that separates the data?

We map the data into a higher dimensional space where it becomes linearly separable, then find a hyperplane there. A linear boundary in the higher dimensional space corresponds to a non-linear boundary in the original space.

Feature Maps:

A feature map $\varphi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ takes a d -dimensional input and maps it to a D -dimensional space where $D \gg d$. For example, for $d=2$ inputs (x_1, x_2) , a degree-2 polynomial feature map might be:

$$\varphi(x) = [x_1^2, x_1 x_2, x_2^2, x_1, x_2, 1]$$

This is a 6 dimensional space. A linear SVM in this space can find quadratic boundaries in the original 2D space. (we find a linear combination of these polynomial terms).

However we run into an issue. Let's say that we're only concerned with degree- n polynomial feature maps. We must find all combinations of each input up to its n^{th} term. For degree 3, $D=10$. For degree 10, $D>100$. D grows quickly with n . Thus, explicitly computing $\varphi(x)$ for every training point and storing it becomes infeasible.

The key observation is that the objective function and predictions depend on the data **only through inner products**.

Dual Objective:

$$\sum_i \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

Prediction:

$$\text{Sign} \left(\sum_i \alpha_i y^{(i)} x^{(i)T} x + b \right)$$

If we applied a feature map, every x would be replaced $\varphi(x)$, and every inner product $x_i^T x_j$ would become $\varphi(x_i)^T \varphi(x_j)$.

The question is: **can we compute $\varphi(x_i)^T \varphi(x_j)$ without computing $\varphi(x_i)$ and $\varphi(x_j)$ explicitly?**

We can, if we can find a function K such that:

$$K(x, x') = \varphi(x)^T \varphi(x')$$

Then we never need $\varphi(x)$ at all, we just replace all the inner products with $K(x_i, x_j)$, and the sum implicitly operates in the feature space without ever going there.

The Polynomial Kernel

As seen earlier, sometimes we want **polynomial feature spaces**. The kernel we use takes the form:

$$K(x, y) = (x^T y + c)^d$$

where c is a constant (usually 1).

For the sake of example let

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

a 2D input space. Let's use $c=1$ and $d=2$

$$K(x, y) = (x^T y + 1)^2$$

$$= (x_1 y_1 + x_2 y_2 + 1)^2$$

$$= x_1^2 y_1^2 + x_2^2 y_2^2 + 1 + 2x_1 x_2 y_1 y_2 + 2x_1 y_1 + 2x_2 y_2$$

Notice that in this sum of terms, each is a product of something involving x only and something involving y only. This is exactly the form of an inner product $\varphi(x)^T \varphi(y)$. By inspection, we see that:

$$\varphi(x) = [x_1^2, x_2^2, 1, \sqrt{2} x_1 x_2, \sqrt{2} x_1, \sqrt{2} x_2]$$

So $K(x, y) = \varphi(x)^T \varphi(y)$, and we never needed to explicitly compute the feature map. Note that in this example we see:

- Degree 2: $x_1^2, x_2^2, x_1 x_2$
- Degree 1: x_1, x_2
- Degree 0: 1

So all the polynomial features are captured.

Thus, generalizing up to degree d :

$$K(x, y) = (x^T y + c)^d$$

Why is the Kernel Trick Valid?

The only change is replacing every inner product

With K . Since inner products determine distances, angles and projections, as long as the kernel computes a valid inner product in some space, all the geometry is preserved.

The RBF Kernel

The Radial Basis Function Kernel is another special kernel. It is defined as

$$K(x, y) = \exp(-\gamma \|x - y\|^2)$$

Where $\gamma = 1/2\sigma^2$

To see why it's so powerful, let's expand it.

$$\|x - y\|^2 = (x - y)^T (x - y)$$

$$= \|x\|^2 - 2x^T y + \|y\|^2$$

So:

$$K(x, y) = \exp(-\gamma (\|x\|^2 - 2x^T y + \|y\|^2))$$

$$= \exp(-\gamma \|x\|^2) \cdot \exp(2\gamma x^T y) \cdot \exp(-\gamma \|y\|^2)$$

The $\exp(-\gamma \|x\|^2)$ terms are constants.

They don't really change the feature map, apart from rescaling. The interesting bit is the middle term. We expand using the Taylor series:

$$\exp(t) = \sum_n \frac{t^n}{n!}$$

$$\exp(2\gamma x^T y) = \sum_n (2\gamma)^n \frac{(x^T y)^n}{n!}$$

This is why RBF is so powerful: each $(x^T y)^n$ term is a degree n polynomial kernel. Summing over all n from 0 to ∞ gives all polynomial features of every degree simultaneously. This means RBF is incredibly flexible, it can accurately

separate highly irregular data. This is because it corresponds to an infinite dimensional feature space.

This is also why the kernel trick is so good. Even though the feature space has infinite dimensions, we can evaluate the kernel in $O(d)$ using the closed form $\exp(-\gamma \|x - y\|^2)$ which only uses a simple inner product.

The Intuition

RBF measures similarity between two points based on their Euclidean distance. Points close together get a kernel value near 1. Points far apart get a value near 0. γ controls how quickly the similarity decays with distance.

So the RBF asks: how similar is the new point to each support vector? The prediction is a weighted combination of these similarities. The new point gets classified based on which support vectors it resembles most.

Multi-Class SVMs

SVMs are inherently binary classifiers.

For problems with K classes, there are two standard approaches:

- One-vs-Rest

Train K binary SVMs, each separating one class from all others. At test time, assign the class whose SVM gives the highest confidence score (largest $w^T x + b$). Train each SVM on the full dataset.

- One vs One

Train $K(K-1)/2$ binary SVMs, one for each pair of classes. At test time, each SVM votes for one class.

and the class with the most votes wins. Trains many SVMs, but each on a smaller dataset - often faster for large k .

SVM vs Other Models

- **Logistic Regression**

SVM is sparse, only support vectors matter, LR is dense - all points matter. LR gives calibrated probabilities, preferred when you need probability estimates. SVM for hard classifications.

- **Decision Trees**

SVMs require feature scaling, trees do not. SVMs are better for high dimensional data with few examples